Unit: Search strategies and problem solving

Problem solving agent

It is a AI agent, that searches for sequence of actions that transforms initial state into goal state.

Characters

→ It is goal oriented
→ It uses search technologies
→ It chooses best path.
→ It slovers well defined problems.

~~→ How~~

✱ How problem solving agent works

① problem formulation
② State space ~~search~~. representation

✱ Problem formulation (define the problem)

→ It means representing any problem in a way that AI agent can ~~so~~ ~~slo~~ solved
ex:- It answers the questions like where i am, where do i want to go, what actions can i perform, which path is best.

✱ components of problem formulation

① initial state (starting position)
② Goal state
③ actions
④ Transiation modle. (It shows what's happens after ~~performing~~ performing an ~~modle~~ option)
⑤ Goal test
⑥ Path cost → measures the total cost of reaching the goal.
→ cost may represent distance, time, money and ~~ener~~ energy.

② State space ~~search~~ representation. (slovers the problem)

After formulating the problem agent must search for solution by ~~explo~~ exploring all possible ~~sto~~ ~~path~~ state ~~cal~~ to ~~find~~ find a path from initial state to Goal state.

state

represents all possible ~~sutiations~~ situation of the problem.

state space

collection all possible states.

Components of state space representation

① states
② initial state
③ Goal state
④ actions (operations
⑤ Transition → conection b/w the state.

Ex:- Root finding. suppose a person wants to travel from A to D.
state → A, B, C, D.
initial state → A
Goal state → D
actions → A→B→D, A→C→D.

advantages

→ clearly represents the problem
→ helps seach algorithms and find solution
→ It can be used in root finding, robo navigation, puzzals and another games

Search techaniques

they are the methods use by AI agent to explore state space and find a sequence of actions that leads from initial state to final state.

They are two types

1) Uninformed	2) Informed.
→ It also called as blind search.	→ It also called heuristic search.
→ Used no additional knowledge about the goal.	→ It uses heuristic information about the goal (estimated)
→ usually explores more nodes	→ usually explore less nodes
→ genarly slow for larger problems	→ genarly faster and more efficent.
→ requires more time and memory	→ require less time & memory
→ simple to implement	→ complex to implement
→ Ex: BFS, DFS, UCS, DLS, ICS,	→ Ex: A*, IDS, greede BFS

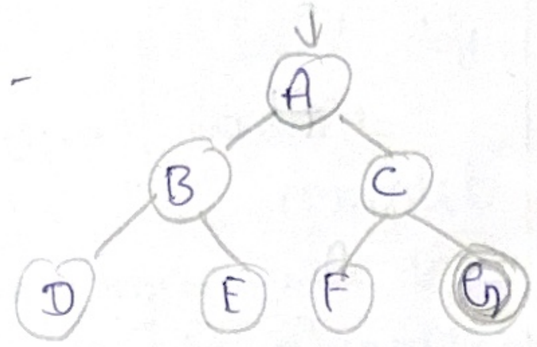
BFS (Breadth first search)

→ It is the Uninformed search algorithem,
1. That explores the search tree level by level,
2. It visits all nodes that the current level
3. before moving to the next level.
4.

Working

5. 1) Start from initial node (root node)
6. 2) Insert the root node into queue (FIFO)
   3) remove the front node from the queue and visit it
   3a) add all its unvisited children nodes to the rear of the queue
   4) repeat step 3 & 4 until goal node is find or queue becomes empty.

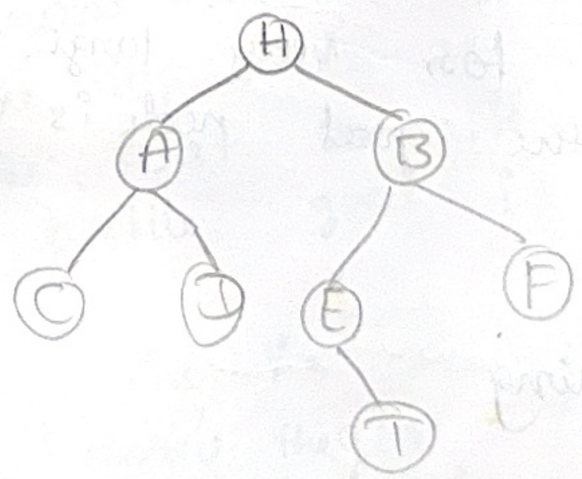
Ex:-



step	queue	visited
start	[A]	-
1	[B C]	A
2	[C D E]	B
3	[D E F G]	C
4	[E F G]	D
5	[F G]	E
6	[G]	F
7	—	G

A → B → C → D → E → F → G. = path.

Ex:- A person start at home H went to reach hospital & slove it by BFS.



steps	queue	visited
1	H	-
2	A B	H
3	B C D	A
4	C D E F	B
5	D E F	C
6	E F	D
7	F T	E
8	T	F
9	-	T

H → A → B → C → D → E → F → T

advantages

→ Find the solution if it is having one exists
→ Finds the shortest path in unvisited graph
→ simple to understand & implement

disadvantages

→ requires larger memory because it stores many nodes.
→ Can be slow for very large data set.
→ Not efficient, when the goal node is deeper.

applications

→ Social networks analises
→ GPS and roof planing
→ Pazzal sloving
→ Search engins

Characters of BFS

→ It is uninformed search
→ It is level by level search.
→ Uses queue
→ It is complete algorithem → It grantes solution if one exists.
→ optimal algorithem → helps to shortest path
→ each nodes visited only once.
→ It high memory require
→ Suitable for shallow goal node.

DFS (Dept first search)

→ It is uninform search algorithem that explores search tree by going as deep as possible along one branch before backs tracking to another branches.

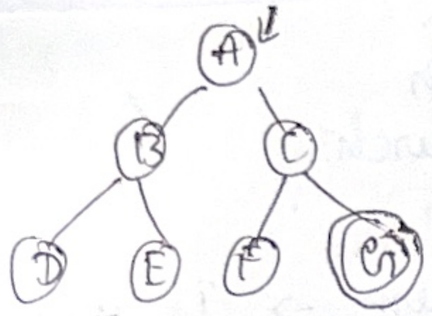
Characters of DFS

→ It uses stack.
→ explores the deepest node first.
→ Back tracks when no more childrens are availabe
→ does not garaenty shortest path.
→ memory usage is low compare to BFS

algorithm / working

1) start from the initial node.
2) Marks the top node as visited.
3) visit the first unvisited child.
4) repeate untile goal is found or no child exists
5) If no child is availabe back track.
6) continue untile the goal is reach. or all nodes are visited

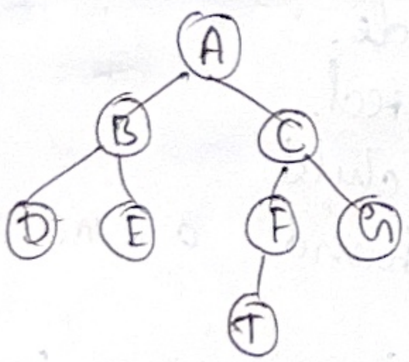
Ex!-



step	stacks	visited
1	A	
2	~~A~~ B C	A
3	E D C	B
4	D C	E
5	C	D
6	F G	C
7	G	F
8	∪	G

A → B → E → D → C → F → G

2)

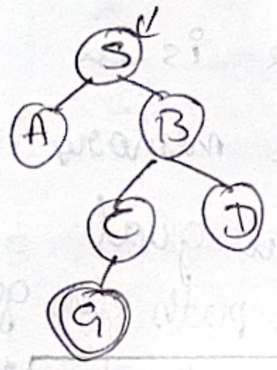


Solve that using DFS

step	stacks	visited
1	A	
2	B C	A
3	D E C	B
4	E C	D
5	C	F
6	F G	C
7	T G	F
8	G	T

A → B → D → E → C → F → T

3)



step	stacks	visited
1	S	
2	A B	S
3	B	A
4	C D	B
5	G D	C
6	D	G.

S → A → B → C → G

advantages
→ uses less memory
→ simple to implement.
→ can quickly reach deep solution
→ suitable when solution is in the deep.

disadvantage
→ does not garenty shortest path
→ can enter infinite path if cycles are not handled

applications
→ Pazzal solving
→ path finding
→ cycle detection
→ back tracking problems

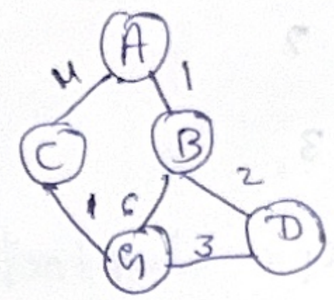
DFS	BFS
→ stack is use	→ queue is use
→ less memory is use	→ more memory is use
→ depth goal	→ shallow goal
→ shortest path not garenty	→ shortest path can garenty
→ it explores the depth first.	→ level by level traversal.

Uniform cost search
It is the uninformed search algorithem. that finds least cost path from a start node to a goal node in a weighted graph It uses preority queue.

algorithem / working
1) insert the start node into the preority queue with cost '0'.
2) remove the node with minimum pathcost
3) If it is goal node written the solution.
4) otherwise explained the node add it's neiber with their

5) repeat until the goal node is find or queue become empty.

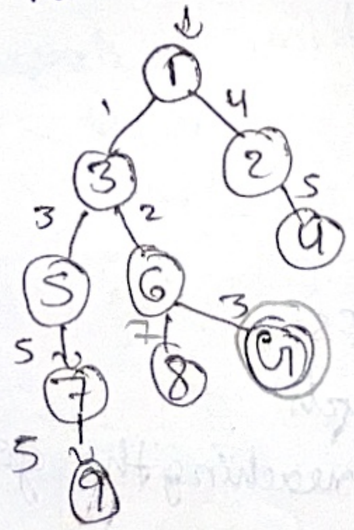
Ex:- start → A
goal → G
Edge and cost
A → B(1)
A → C(4)
B → D(2)
B → G(6)
C → G(1)
D → G(3)



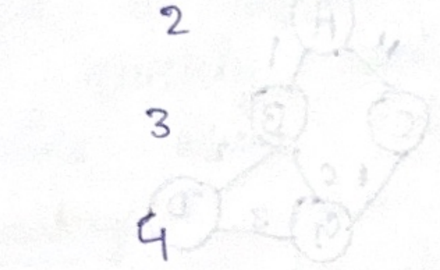
step	Expanded node	Priority queue
1		A(0)
2	A(0) →	B(1), C(4)
3	B(1) →	D(3) G(8) C(4)
4	D →	C(4) G(6), G(7)
5	C →	G(5), G(6), G(7)
6	G →	

A → B → D → C → G → UCS Traversal path.
A → C → G - optimal path
4+1=5

Characters of UCS
→ It is complete solution
→ It is optimal.
→ It is uninformed search algorithem
→ No touristic function.

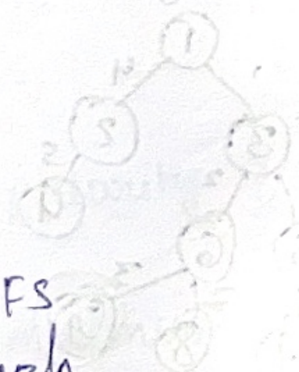


step	expanded node	Priority queue
1		1
2	1	3 2
3	3	6(7) 5(4) 9 (4)
4	6	8 9 5(4) 2(4) 4(6) 8(10)
5	5	2(4) 5 4(6) 8
6	2	5 4(6) 8 4
7	5	4(6) 8 4 7
8	4	8 4 7



1		
2	1	3(1), 2(4)
3	3	6(5) 5(4) 2(4)
4	6	5(4) 2(4) 4(6) 8(10)
5	2	5(4) 4(6) 4(9) 8(10)
6	5	4(6) 9(9) 4(9) 8(10)
7	4	

1 → 3 → 6 → 9 → 5 → 4



advantages

- It finds the optimal solutions
- works with edge cost
- It is complete

disadvantages

- ~~x~~ finds the optimal path ~~x~~
- It require more memory then DFS
- Can be slow for very large graph
- May expands many nodes before reaching the goal

applications

- navigation in root
- Robot path planing
- AI gaming

$f(n) = g(n)$

$f(n)$ evalution function

$g(n)$ cumulative path cost from start node to current node

⊛ Greedy best first search

It is the a informed search algorithem that selects the node that appears to be closest to the goal using ~~to~~ heuristic function

→ It always expands the node with smallest heuristic value, It is not cumulative

→ Formula.

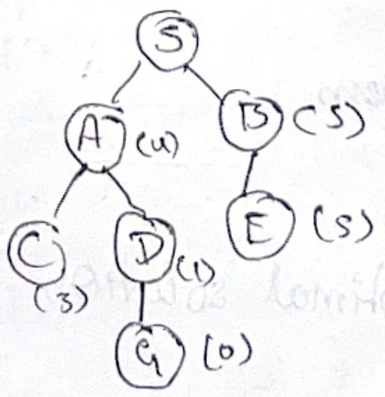
$f(n) = h(n)$

$h(n)$ → heuristic estimate (estimated distance from current ~~goal~~ to goal node)

algorithem / working

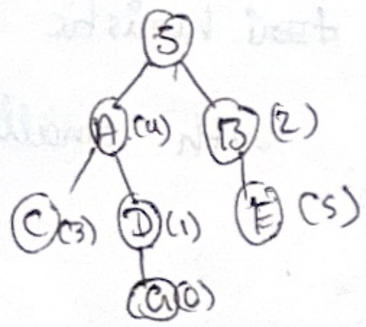
1) insert the initial node into the priority queue
2) calculate the heuristic value, $h(n)$
3) remove node with smallest $h(n)$
4) stop if it is the goal node
5) other wise expand it's childeren
6) repeat until the goal is found or queue become emtye

Ex-



step	priority queue	expanded node
1	S	
2	A(4) B(5)	S
3	D(1) C(3) B(5)	A
4	g(0) C(3) B(5)	D
5	C(3) B(5)	g

S → A → D → g → Traversal path.



step	priority queue	expanded node
1	S	
2	B(2) A(4)	S
3	A(4) E(5)	B
4	D(1) C(3) E(5)	A
5	G(0) C(3) E(5)	D
6	C(3) E(5)	G

S → B → A → D → G → Traversal path

S → A → D → G - optimal path

Characters

→ It is informed search algorithm.
→ It uses heuristic value
→ It uses priority queue
→ It does not always finds the optimal solution

advantages

→ faster compare to uninformed algorithem
→ It uses heuristic value.
→ simple to implement.

~~disadvantages~~ applications

→ path finding
→ robot navigation
→ AI gaming
→ pazzal sloving

disadvantages

→ If not always complete.
→ does not garenty shortest path.
→ can explore unnessery paths if heuristic value is misgleding.

A* ~~alog~~ algorithem.

A* ~~alg~~ is the inform search algorithem used to find least cost path from a start node to the goal node.

formula : $f(n) = g(n) + h(n)$.

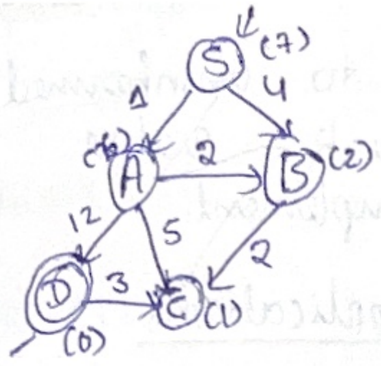
f(n) => Total estimated cost.
g(n) => actual path cost from the start to the current node
h(n) => Estimated cost from current node to goal node

algorithem,

1) start with the initial node.
2) Calculate $f(n) = g(n) + h(n)$.
3) Insert start node into the priority queue
4) select and remove the node having smallest f(n) from the priority queue
5) If the selected node is goal node than stop.
6) another wise expand the node and generate its neighber nodes => insert the new node into priority queue.
7) repeate the set 4 from 4 to 8 until the goal node is found.

Ex-

node	g(n)	h(n)
S→A	1	S→7
S→B	4	A-6
A→B	2	B-2
A→D	12	C-1
A→C	5	D-0
C→D	3	
B→C	2	



step	currentnode	f(n)	priority queue
			A(7) ; B(6)
1	S		
2	B	6	A(7) C(7)
3	A	7	B(5) C(7) D(13)
4	B	5	C(6) D(13)
5	C	7	D(13) D(9) C(6)
6	D	9	D(13)

path: S→B→A→B→C→D

step	current node	f(n)	priority queue
			A(7) ; B(6)
1	S		
2	B	6	A(7), C(7)
3	A	7	B(5) C(7) D(13) C(7)
4	B	5	C(7) C(6) D(13) C(7)
5	C	6	D(8) C(7) D(13) C(7)
6	C	7	D(8) D(13) C(7) D(9)
7	C	7	D(8) D(9) D(13) D(9)
8	D	8	

path S→A→B→C→D

f(n) = g(n) + h(n)

f(B) = 4 + 2 = 6

f(A) = 1 + 6 = 7

f(C) = 6 + 1 = 7

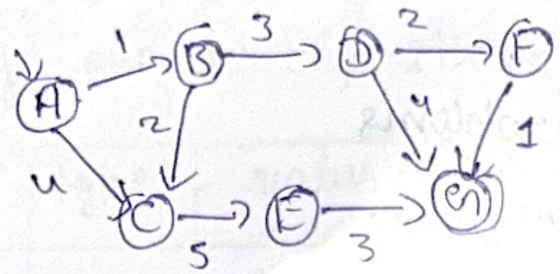
f(C) = 6 + 1 = 7

f(D) = 13 + 0 = 13

f(B) = 3 + 2 = 5

f(D) = 4+2+3+0 = 9

2)



A-5
B-6
C=4
D-3
E=3
F=1
G=0

step	currentnode	f(n)	priority queue
			B(7) C(8)
1	A		
2	B	7	C(8) C(7) D(7)
3	C	7	C(8) D(7) E(11)
4	D	7	C(8) E(11) F(7) G(8)
5	F	7	C(8) E(11) G(7) G(8)
6	G	7	

path: A→B→D→F→G
1 + 3 + 2 + 1 = 7
minimum cost is 7

Characters of A* algorithem.

- informed search
- formula used is f(n) = g(n) + h(n)
- used priority queue data structure is used
- It is optimal algorithem.
- It is the complete algorithem

advantages:
-> minimum cost path is finds used it.
-> It is faster than uninformed algorithem.
-> It is goal oriented
-> It is complete and flexible.

disadvantages
-> depends on heuristic.
-> slow for larger problems
-> memory is high requires
-> can explore more nodes unnessry
-> implemention is complex

applications
-> GPS ~~nati~~ navigation
-> maize sloving
-> pazzal sloving
-> Robotics

Game playing in AI
-> It is use of AI techanic to make computer play a games intelgently against human or another computer.

Common techanique
-> Minmax algorithem
-> Alpha beta praning

Min max algorithem

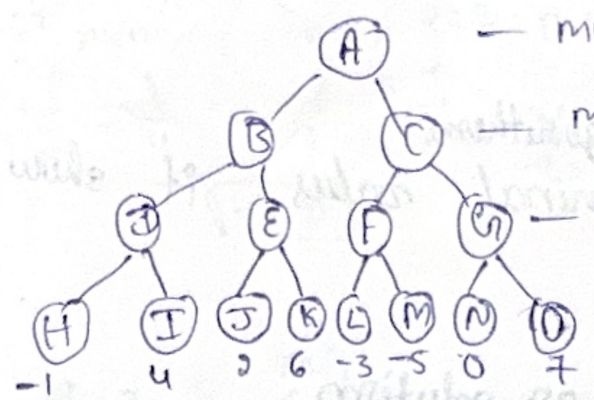
It is the AI algorithem, used for decision making in two player games.
-> It mainly used in game playes there are two players.
-> One player try to maximize the score and another player try to minimize the maximum score.

Ex: chess, tik tak

Note: min = $-\infty$
max = ∞

-> The two player are called max min.

Ex1



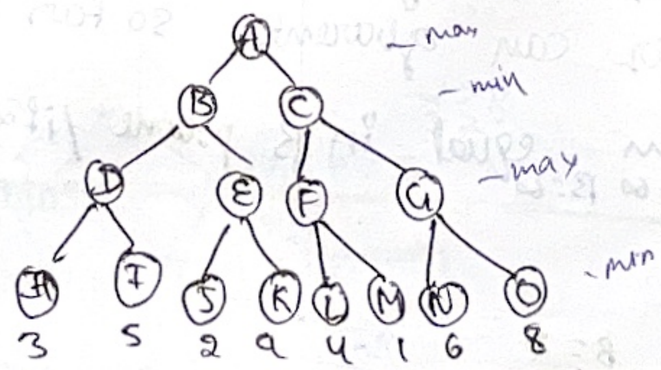
H	-1
I	4
J	2
K	6
L	-3
m	-5
N	0
O	7

slove using min max algorithem.

step	node	type	values considered	result
1	D	max	max(-1, 4)	4
2	E	max	max(2, 6)	6
3	F	max	max(-3, -5)	-3
4	G	max	max(0, 7)	7
5	B	min	min(4, 6)	4
6	C	min	min(-3, 7)	-3
7	A	max	max(4, -3)	4

Max=4
path:- A -> B -> D -> I = 4

Ex:-



step	node	type	values considered	result
1	D	max	max (3, 5)	5
2	E	max	max (2 9)	9
3	F	max	max (4, 1)	4
4	G	max	max (6, 8)	8
5	B	min	min (5, 9)	5
6	C	min	min (4, 8)	4
7	A	max	max (5, 4)	5

Path: A -> B -> D -> I = 5

Character of min max algorithem.

- ⇒ It is a two players game.
- → It is a tree based algorithem
- → It should have a terminal values if show final game states
- → levels are alternative
- → keeps the optimal decision or solution.
- → inteligent opponent
- → It uses DFS
- → It use back tracking.
- → It is complete algorithem
- ~~Time com~~

Dis advantages

- → It is slove for complex games such as chess

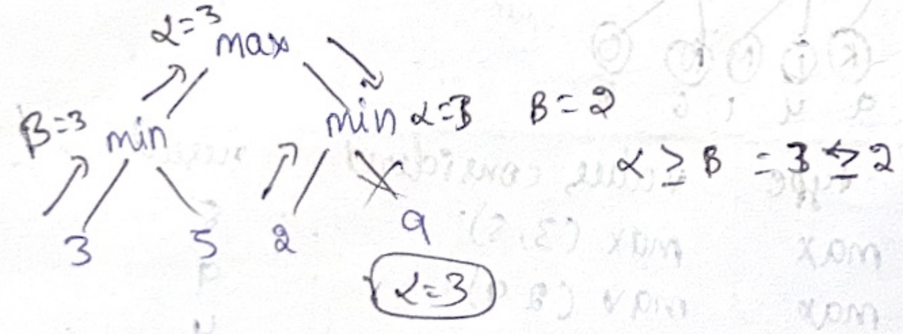
Alpha beta Pruning

It is a optimization techanic for minmax algorithem, used in two player games like chess, tick tacke

- → It reduce the number of notes evaluated in the game tree without affeting the final decision. note
- α → Best value Max player can guarentee so far
- β → Best value min player can guarentee so far
- → If alpha greater than equal in β prune / if α≥β prune

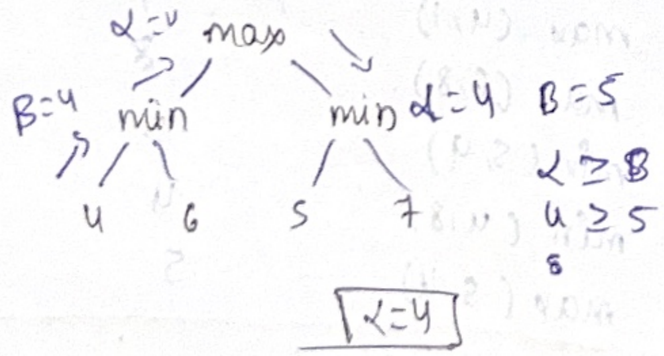
α=-∞ β=+∞

Ex.

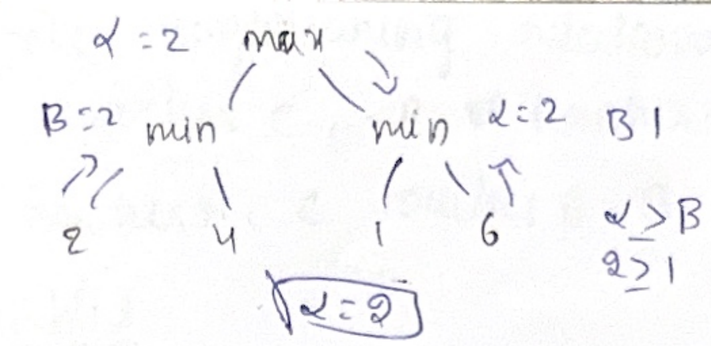


α≥B 3≥2

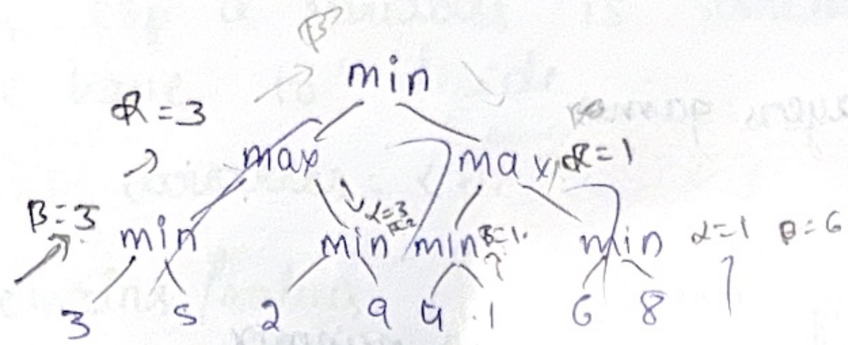
Ex

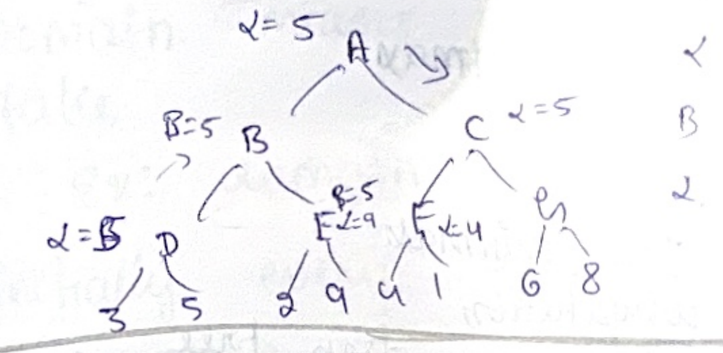


α≥B 4≥5



α≥B 2≥1

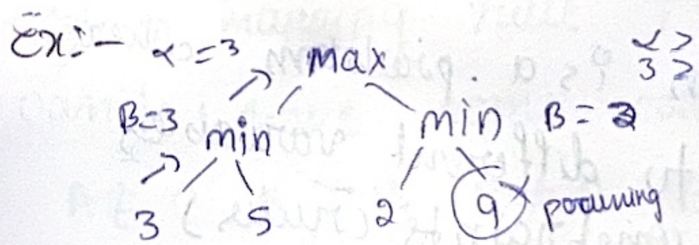




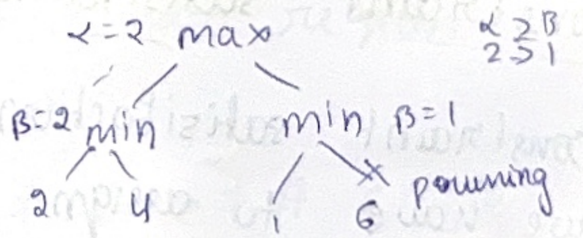
Characters

- → It reduce the number of node being check.
- → It prunes the nodes based on the condition α≥B.
- → It is complex than min max but take less time than min max
- → It is a optimize version of a minmax algorithem

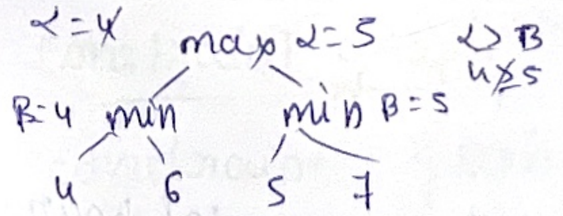
Ex:-



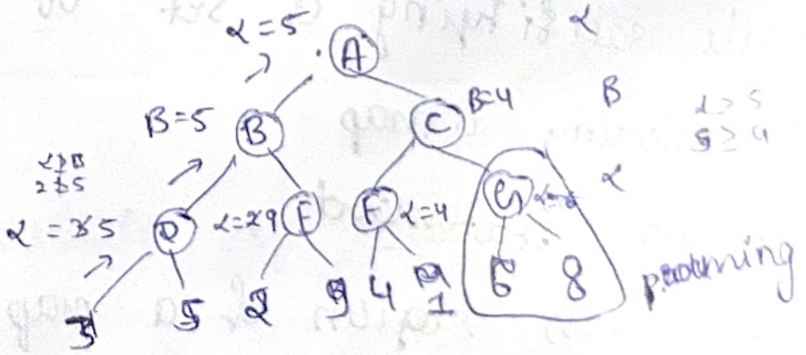
α≥B 3≥9

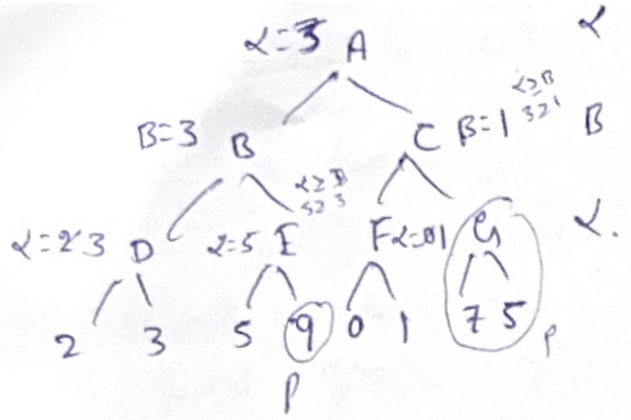


α≥B 2≥1



α≥B 4≥5





-> works only on two players games

advantages

-> reduce the nodes being check
-> produce the same optimal result as minmax
-> It is more effective than minmax

disadvantages

-> It is complex algorithem than minmax
-> ~~stile~~ stile requires significate compitation
-> depth ~~immutation~~ limitition: search very deep tree can be time consuming

applications

-> chess
-> tick tocko
-> checkers

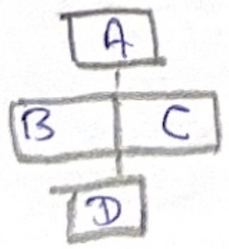
Constraint Satisifaction problem

Constraint satisifaction problem is a problem where we have to assign values to different variables while satisfying a set of constraints (rules)

Ex:- coloring a map

problem statement

color each region of a map so that two neighbouring region do not have same color



suppose neighbouring relationship are A touches B A touches C, B ~~st~~ touches C, B touches D, C touches D B touches A, C touches A B & D, D touches B C.

variables

In csp a variables is something whose values we have to decide

Ex:- variables = {A, B, C, D}

Domains /values

Domain means possible values the variable can take

Ex:- domain = { Red, green, blue}

initially every region can use ~~any~~ any colour.

A = {R, G, B}
B = {R, G, B}
C = {R, G, B}
D = {R, G, B}

Constraints (rules that must be followed)

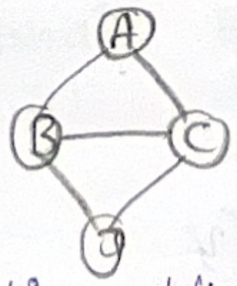
Rules that must be followed

Colour mapping rule is two neighbour region can't have some colours

$A \neq C$ $B \neq B$ $B \neq A$ $B \neq C$ $B \neq D$ $C \neq A$ $C \neq D$
$A = D$

Constraint graph

constraint graph represents csp visually.



so there is no line b/w A and D b/w they do not have a constraint.

forward checking

neighbours

A → B, C
B → A, C, D
C → A, B, D
D ⇒ B, C.

Available colours : {Red Green Blue}

step1 : Initial domains before assigning anything

Region	domain
A	{R, G, B}
B	{R, G, B}
C	{R, G, B}
D	{R, G, B}

step 2: Choose A: red. Now use forward checking

A is connected to B and C

∴ B, C. can't use Red

Remove red from their domain

Region	domain
A	Red
B	{G, B}
C	{G, B}
D	{R, G, B}

notice D is unchanged

Now B : {green, blue}

step 3: assign B: green

B is connected to A, C, D

A is already red after forward checking

Region	domain
A	Red
B	green
C	{blue}
D	{Red, blue}

step4:- Now C has only blue so, C= Blue C connected to A, C, D

A is red, B is green.

Region	domain
A	Red
B	green
C	Blue
D	{Red}

step5:- assign D: Red

A =	Red
B	green
C	Blue
D	Red

Forward checking:- Forward checking is a csp technique in which after assigning a value to the variable invalid values are immediately removed from the domains of its neighbouring unassigned variable

Exam time table

Suppose we have 3 example maths, science, english available time slots 11 am, 9am, 2pm constraints maths & science can't be at the same time

region	value/domain
maths	{11am, 9am, 2pm}
science	{9, 11, 2PM}
English	{9, 11, 2pm}

constraints

S ≠ M , M ≠ S

Constraint graph

M — S — E

step 1 : initial domains before assigning anytime

Region	domain
maths	{9am, 11am, 2pm}
science	{9am, 11am 2pm}
English	{9am, 11am, 2pm}

step2: Choose maths = 9am, now use forward checking
maths is connected to science.
∴ science can't use time slot 9am

Region	Domine
maths	9am
science	{11am, 2pm}
English	{9am, 11am, 2pm}

Notice english is unchanged
Now science = {11am, 2pm}

step 3: assign science = 11am
science is connected to both maths & english.
maths is already 9am after forward checking.

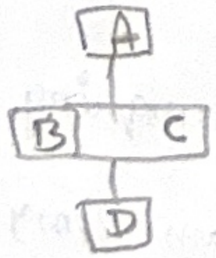
Region	Domain
maths	9am
science	11am
English	2pm.

Step4: Now english has only 2 pm so english = 2pm
English is connected to science

Region	Domain
maths	9am
science	11am
English	2pm

Backward checking

Backward checking mean: checking newly assigned value against the already assigned neighbours variables to make ~~same~~ sure there is no constraint valiation.



A = D

A = {R, G, B}
B = {R, G, B}
C = {R, G, B}
D = {R, G, B}

A ≠ B
A ≠ C
B ≠ C
B ≠ A
C ≠ A
C ≠ B
C ≠ D

step 1: A = Red

step 2: A = Red ✓
B = Red ✗
B = green

step 3: A = Red ✓
C = Red ✗
B = green ✓
C = green ✗
C = Blue

step 4: A = Red ✓ A = D
D = Red
D = Red

A = Red
B = green
C = Blue
D = Red.